TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BÀI TIỂU LUẬN

MÔN HỌC

KỸ THUẬT LẬP TRÌNH NÂNG CAO

GIÁO VIÊN HƯỚNG DẪN : TS. NGHIÊM VĂN TÍNH

HỌ VÀ TÊN SINH VIÊN : NGUYỄN PHÚ HƯNG

LỚP : K59.KMT.K01

MSSV : K235480106091

THÁI NGUYÊN – 2026

TRƯỜNG ĐHKTCN
KHOA ĐIỆN TỬ

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

BÀI TIỂU LUẬN

MÔN HỌC KỸ THUẬT LẬP TRÌNH NÂNG CAO

BỘ MÔN CÔNG NGHỆ THÔNG TIN

Sinh viên: Nguyễn Phú Hưng *Khóa: 2023-2028*

Lớp: K59.KMT.K01 *Ngành: Kỹ thuật máy tính*

Giáo viên hướng dẫn: TS. Nghiêm Văn Tính

Ngày giao đề 30/09/2026 *Ngày hoàn thành / /2026*

Tên đề tài: Xây dựng chương trình quản lý lịch làm việc cá nhân
Yêu cầu thực hiện:
- Yêu cầu về lý thuyết: Trình bày cơ sở lý thuyết liên quan, phân tích bài toán, thuật toán sử dụng, đánh giá độ phức tạp.
- Yêu cầu về thực nghiệm/kết quả: Xây dựng chương trình hoàn chỉnh bằng một ngôn ngữ cụ thể (C/C++, Java hoặc Python), có giao diện người dùng thân thiện; chạy thử nghiệm với dữ liệu mẫu và trình bày kết quả.

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày….tháng…..năm 20....

GIÁO VIÊN HƯỚNG DẪN

(Ký ghi rõ họ tên)

MỤC LỤC

LỜI NÓI ĐẦU 2
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH BÀI TOÁN 3
1.1. Đặt vấn đề và mục tiêu 3
1.2. Phân tích bài toán 3
1.3. Cơ sở lý thuyết 5
CHƯƠNG 2: THUẬT TOÁN VÀ ĐÁNH GIÁ ĐỘ PHỨC TẠP 7
2.1. Phát hiện các cặp việc trùng giờ 7
2.2. Tìm khoảng thời gian rảnh 8
2.3. Sắp xếp thứ tự thực hiện bằng thuật toán Kahn 10
2.4. Lập lịch khoảng có trọng số bằng quy hoạch động 11
2.5. Gỡ trùng lịch tự động 14
2.6. Phân làn khi vẽ lịch tuần 14
2.7. Tổng hợp độ phức tạp 15
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG CHƯƠNG TRÌNH 16
3.1. Ngôn ngữ và công cụ 16
3.2. Kiến trúc chương trình 16
3.3. Thiết kế lớp dữ liệu 16
3.4. Thiết kế cơ sở dữ liệu 17
3.5. Xử lý khi lưu công việc 17
3.6. Thiết kế giao diện 18
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 21
4.1. Môi trường và phương pháp 21
4.2. Bộ dữ liệu mẫu 21
4.3. Kết quả chạy trên dữ liệu mẫu 22
4.4. Kiểm tra tính đúng đắn 26
4.5. Đánh giá hiệu năng 27
4.6. Đánh giá chung và hạn chế 32
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 33
1. Kết luận 33
2. Hướng phát triển 33
TÀI LIỆU THAM KHẢO 34
PHỤ LỤC: HƯỚNG DẪN CHẠY CHƯƠNG TRÌNH 35

LỜI NÓI ĐẦU

Mỗi sinh viên, mỗi người đi làm đều phải sắp xếp nhiều việc trong cùng một khoảng thời gian có hạn: giờ học, cuộc họp, bài tập có thứ tự trước sau và cả thời gian cá nhân. Các ứng dụng lịch thông thường giúp ghi lại và nhắc việc, nhưng khi lịch bị trùng thì người dùng vẫn phải tự quyết định giữ việc nào và dời việc nào sang lúc nào.

Về bản chất, những câu hỏi đó là các bài toán kinh điển của khoa học máy tính: phát hiện giao nhau của các khoảng, sắp xếp topo trên đồ thị có hướng không chu trình và lập lịch khoảng có trọng số. Vì vậy em chọn đề tài **"Xây dựng chương trình quản lý lịch làm việc cá nhân"** để vận dụng kiến thức môn Kỹ thuật lập trình nâng cao vào một sản phẩm có thể dùng hằng ngày.

Chương trình được viết bằng Python, giao diện đồ họa Tkinter, lưu trữ bằng SQLite. Khác với một ứng dụng chỉ có các chức năng thêm, sửa, xóa, chương trình trả lời được bốn câu hỏi: lịch có bị trùng không, còn giờ rảnh nào phù hợp, nên làm việc theo thứ tự nào, và khi không thể làm hết thì giữ việc nào, dời việc còn lại đi đâu.

Báo cáo gồm bốn chương. Chương 1 phân tích bài toán và trình bày cơ sở lý thuyết. Chương 2 trình bày từng thuật toán kèm lập luận về tính đúng đắn và độ phức tạp. Chương 3 mô tả thiết kế và xây dựng chương trình. Chương 4 trình bày kết quả chạy trên dữ liệu mẫu, kết quả đối chiếu với nghiệm vét cạn và số đo hiệu năng, đồng thời chỉ ra những điểm số đo chưa khớp lý thuyết và các hạn chế còn lại.

Em xin gửi lời cảm ơn đến thầy **TS. Nghiêm Văn Tính** đã giảng dạy và hướng dẫn em trong suốt quá trình học tập môn học. Do kinh nghiệm còn hạn chế, báo cáo không tránh khỏi thiếu sót, em rất mong nhận được nhận xét của thầy để hoàn thiện hơn.

Em xin chân thành cảm ơn!

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH BÀI TOÁN

1.1. Đặt vấn đề và mục tiêu

Khi quản lý lịch làm việc cá nhân, người dùng thường gặp bốn khó khăn: hai việc bị xếp trùng giờ mà không được cảnh báo; không biết khoảng trống nào đủ dài cho một việc mới; làm sai thứ tự giữa các việc có quan hệ trước sau; và lúng túng khi không thể làm hết mọi việc đã xếp.

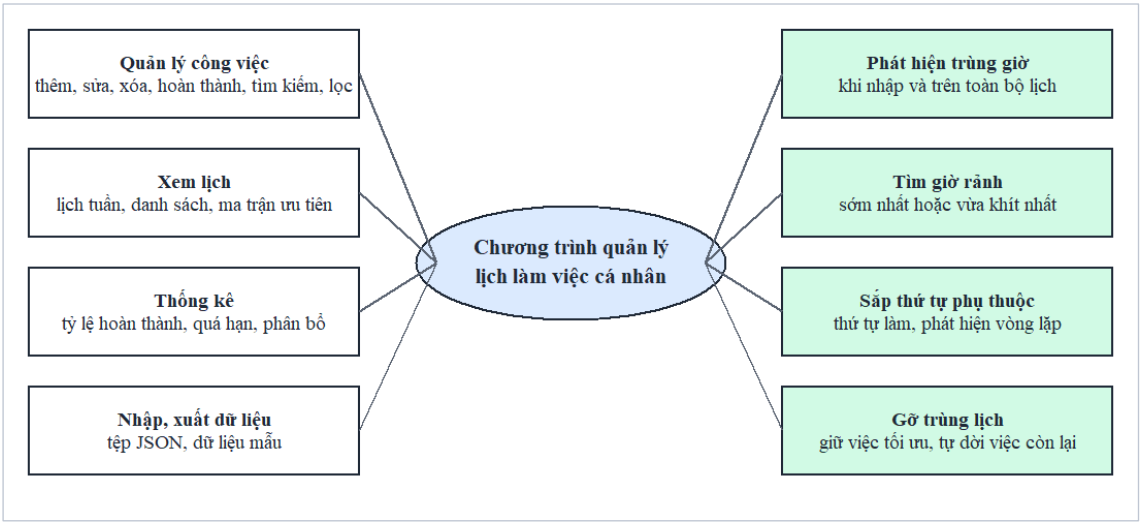
Từ đó, đề tài đặt ra các mục tiêu sau:

- Xây dựng chương trình hoàn chỉnh bằng Python, có giao diện đồ họa thân thiện, lưu dữ liệu bền vững.
- Mô hình hóa từng khó khăn thành một bài toán thuật toán, chọn thuật toán phù hợp, lập luận tính đúng đắn và đánh giá độ phức tạp.
- Kiểm chứng bằng thực nghiệm: chạy trên dữ liệu mẫu, đối chiếu với nghiệm vét cạn và đo hiệu năng theo quy mô dữ liệu.

1.2. Phân tích bài toán

1.2.1. Yêu cầu chức năng

Hình 1 tóm tắt các nhóm chức năng của chương trình. Nhóm bên trái là các chức năng quản lý thông thường, nhóm bên phải là các chức năng dựa trên thuật toán, đây là phần trọng tâm của đề tài.



Hình 1. Các nhóm chức năng của chương trình

1.2.2. Yêu cầu phi chức năng

- **Phản hồi nhanh:** với lịch cá nhân vài nghìn việc, các thao tác không gây trễ dễ nhận thấy.

- **Dễ dùng:** chọn ngày trên lịch tháng thay vì gõ theo định dạng, lỗi được báo ngay tại chỗ nhập.
- **Toàn vẹn dữ liệu:** không lưu phụ thuộc sai; tệp nhập lỗi không làm thay đổi dữ liệu đang có.
- **Không phụ thuộc thư viện ngoài:** chỉ dùng thư viện chuẩn của Python nên dễ cài đặt, dễ chạy lại.

1.2.3. Mô hình hóa dữ liệu

Mỗi công việc i được biểu diễn bởi thời điểm bắt đầu s_i, thời điểm kết thúc e_i (với $e_i > s_i$), thời lượng $d_i = e_i - s_i$, mức ưu tiên p_i thuộc {1, 2, 3, 4} (Thấp, Vừa, Cao, Khẩn cấp), trạng thái, danh mục và tập Dep_i các việc phải xong trước.

Thời gian của một việc được xem là **khoảng nửa mở** $[s_i, e_i)$. Nhờ đó, một việc kết thúc lúc 10:00 và việc khác bắt đầu đúng 10:00 không bị coi là trùng, đúng với cách hiểu thông thường.

Một việc được gọi là **còn hiệu lực** khi ở trạng thái Chưa bắt đầu hoặc Đang thực hiện. Chỉ các việc còn hiệu lực mới được xét khi phát hiện trùng giờ và khi tối ưu, vì việc đã xong hoặc đã hủy không còn cần giành chỗ. Riêng khi tìm giờ rảnh, việc đã hoàn thành vẫn tính là bận vì khoảng thời gian đó đã được dùng, chỉ việc đã hủy mới trả lại chỗ.

1.2.4. Quy các câu hỏi của người dùng về bài toán thuật toán

Bảng 1 cho thấy mỗi câu hỏi thực tế tương ứng với một bài toán và thuật toán nào. Ký hiệu N là số công việc, K là số cặp trùng giờ, V và E là số đỉnh và số cạnh của đồ thị phụ thuộc.

Bảng 1. Quy câu hỏi của người dùng về bài toán thuật toán

Câu hỏi của người dùng	Bài toán	Thuật toán	Độ phức tạp thời gian
Lịch có việc nào trùng giờ không?	Liệt kê các cặp khoảng giao nhau	Sắp xếp rồi quét, dừng sớm	O(N log N + K)
Còn khoảng trống nào đủ dài?	Phần bù của hợp các khoảng	Gộp khoảng, lấy phần bù	O(N log N)
Làm việc theo thứ tự nào?	Sắp xếp topo trên DAG	Kahn	O(V + E)
Không làm hết thì giữ việc nào?	Lập lịch khoảng có trọng số	Quy hoạch động	O(N log N)
Dời các việc còn lại đi đâu?	Xếp việc vào khe rảnh	Tham lam kết hợp tìm khe	O(D·H·N log N)
Vẽ các việc trùng giờ thế nào?	Phân hoạch khoảng	Tham lam với min-heap	O(N log N)
Việc nào nên làm trước?	Phân loại theo hai tiêu chí	Ma trận Eisenhower	O(N)

1.3. Cơ sở lý thuyết

1.3.1. Điều kiện giao nhau của hai khoảng

Hai khoảng nửa mở $[s_a, e_a)$ và $[s_b, e_b)$ có điểm chung khi và chỉ khi:

$$\max(s_a, s_b) < \min(e_a, e_b)$$

Chứng minh. Một thời điểm t thuộc cả hai khoảng khi $t \geq s_a$, $t \geq s_b$, $t < e_a$ và $t < e_b$, tức là $\max(s_a, s_b) \leq t < \min(e_a, e_b)$. Một giá trị t như vậy tồn tại khi và chỉ khi $\max(s_a, s_b) < \min(e_a, e_b)$. Khi đó phần trùng chính là khoảng $[\max(s_a, s_b), \min(e_a, e_b))$, độ dài của nó là số phút trùng hiển thị cho người dùng.

1.3.2. Độ phức tạp phụ thuộc đầu ra

Bài toán liệt kê mọi cặp trùng giờ có một đặc điểm: bản thân kết quả có thể rất lớn. Nếu tất cả N việc cùng trùng nhau thì có $N(N-1)/2$ cặp, nên không thuật toán nào liệt kê được nhanh hơn $O(N^2)$ trong trường hợp đó. Vì vậy, thay vì chỉ nói theo N, cần đánh giá theo cả N và kích thước đầu ra K. Cách đánh giá này gọi là độ phức tạp phụ thuộc đầu ra (output-sensitive) và sẽ được dùng ở mục 2.1.

1.3.3. Đồ thị có hướng không chu trình và sắp xếp topo

Quan hệ "cần xong trước" được biểu diễn bằng đồ thị có hướng G = (V, E), trong đó mỗi đỉnh là một công việc và cạnh (u, v) nghĩa là u phải xong trước v. Một **thứ tự topo** là cách xếp các đỉnh thành dãy sao cho mọi cạnh đều đi từ đỉnh đứng trước tới đỉnh đứng sau. Thứ tự topo tồn tại khi và chỉ khi G không có chu trình

(DAG). Về mặt ý nghĩa, chu trình là tình huống bế tắc: A chờ B, B lại chờ A, nên không việc nào bắt đầu được [1], [2].

1.3.4. Quy hoạch động

Quy hoạch động áp dụng được khi bài toán có **cấu trúc con tối ưu** (nghiệm tối ưu được tạo từ nghiệm tối ưu của các bài toán con) và **các bài toán con chồng lấp**. Thay vì tính lại nhiều lần, ta lưu kết quả từng bài toán con vào bảng. Bài toán lập lịch khoảng có trọng số là ví dụ điển hình: vét cạn phải xét 2^N tập con, trong khi quy hoạch động chỉ cần N bài toán con [2].

1.3.5. Thuật toán tham lam

Thuật toán tham lam tại mỗi bước chọn phương án tốt nhất trước mắt và không xét lại. Cách này cho nghiệm tối ưu với một số bài toán như phân hoạch khoảng (mục 2.6), nhưng không đúng với mọi bài toán. Ở mục 2.4, em đưa ra phản ví dụ cho thấy tham lam không tối ưu với bài toán có trọng số, và số đo cụ thể ở mục 4.5.3.

1.3.6. Ma trận Eisenhower

Ma trận Eisenhower (được phổ biến qua ma trận quản lý thời gian của Stephen Covey [4]) chia công việc theo hai tiêu chí độc lập là mức quan trọng và mức gấp. Trong chương trình, việc **quan trọng** là việc có ưu tiên từ Cao trở lên, việc **gấp** là việc bắt đầu trong vòng 24 giờ tới hoặc đã qua giờ bắt đầu. Do mô hình dữ liệu chưa có trường hạn chót riêng nên mức gấp được tính theo giờ bắt đầu.

1.3.7. Các cấu trúc dữ liệu sử dụng

Bảng 2. Cấu trúc dữ liệu và thao tác chính trong Python [5], [6]

Cấu trúc dữ liệu	Dùng để	Độ phức tạp thao tác chính
`list` đã sắp xếp + `bisect`	Tìm p(j) trong quy hoạch động	Tìm kiếm nhị phân O(log N)
`dict` (bảng băm)	Tra công việc theo mã, bậc vào, danh sách kề	Tra cứu trung bình O(1)
`collections.deque`	Hàng đợi của thuật toán Kahn	Thêm, lấy ở hai đầu O(1)
`heapq` (min-heap)	Tìm làn rảnh sớm nhất khi vẽ lịch	Thêm, lấy nhỏ nhất O(log N)
Chỉ mục B-tree của SQLite	Truy vấn theo giờ, trạng thái, danh mục	Tìm kiếm O(log N)

CHƯƠNG 2: THUẬT TOÁN VÀ ĐÁNH GIÁ ĐỘ PHỨC TẠP

Các đoạn mã trong chương này được trích nguyên văn từ tệp `core/scheduler_engine.py` (đã lược bỏ chú thích đầu hàm). Các ví dụ dùng bộ dữ liệu mẫu 11 công việc được giới thiệu ở mục 4.2.

2.1. Phát hiện các cặp việc trùng giờ

2.1.1. Thuật toán

Cách đơn giản nhất là xét mọi cặp, tốn $O(N^2)$ phép so sánh dù lịch hầu như không trùng. Chương trình sắp xếp các việc theo giờ bắt đầu rồi duyệt theo ý tưởng đường quét [3]: với mỗi việc i, chỉ xét các việc đứng sau và dừng ngay khi gặp việc bắt đầu từ lúc i kết thúc trở đi.

```
def detect_all_conflicts(tasks: List[Task]) -> List[ConflictResult]:
    if len(tasks) < 2:
        return []

    # Chỉ xét các công việc còn phải làm
    valid_tasks = [t for t in tasks if t.is_active]
    # Sắp xếp theo start_time
    sorted_tasks = sorted(valid_tasks, key=lambda t: t.start_time)

    conflicts: List[ConflictResult] = []
    n = len(sorted_tasks)

    for i in range(n):
        t1 = sorted_tasks[i]
        for j in range(i + 1, n):
            t2 = sorted_tasks[j]
            # Nếu start_time của t2 >= end_time của t1 thì không thể xung đột với t1 và các
task sau đó
            if t2.start_time >= t1.end_time:
                break
            # Ngược lại, vì t2.start_time < t1.end_time và t2.start_time >= t1.start_time =>
có giao thoa
            conflicts.append(ConflictResult(t1, t2))

    return conflicts
```

2.1.2. Tính đúng đắn

Xét hai việc a, b còn hiệu lực bất kỳ, với a đứng trước b trong dãy đã sắp, nên $s_a \le s_b$. Theo mục 1.3.1, chúng trùng nhau khi và chỉ khi $s_b < \min(e_a, e_b)$; do $s_b < e_b$ luôn đúng nên điều kiện chỉ còn là $s_b < e_a$.

- **Không bỏ sót:** nếu $s_b < e_a$ thì mọi việc nằm giữa a và b cũng có giờ bắt đầu không lớn hơn s_b, tức là nhỏ hơn e_a. Vòng lặp trong không dừng trước khi tới b, nên cặp (a, b) được ghi nhận.
- **Không báo sai:** một cặp chỉ được ghi nhận khi điều kiện dừng không thỏa, tức $s_b < e_a$, đúng là điều kiện trùng.

– **Không lặp:** mỗi cặp chỉ được xét với chỉ số i < j, nên được ghi nhận đúng một lần.

2.1.3. Độ phức tạp

Sắp xếp tốn O(N log N). Trong vòng lặp trong, mỗi lần lặp hoặc tạo ra một cặp, hoặc dừng vòng lặp. Tổng số lần tạo cặp là K, số lần dừng tối đa là N, nên phần quét tốn O(N + K). Vậy:

$$T(N) = O(N \log N + K), \quad S(N) = O(N + K)$$

Cần lưu ý rằng K có thể đạt N(N − 1)/2, nên trường hợp xấu nhất vẫn là $O(N^2)$. Thuật toán chỉ thực sự nhanh hơn khi lịch thưa, đúng với lịch cá nhân. Thực nghiệm ở mục 4.5.1 xác nhận cả hai trường hợp này. Trong hộp thoại thêm việc, chương trình dùng hàm `detect_conflicts_for_task` chỉ so một việc với toàn bộ danh sách, tốn O(N).

2.2. Tìm khoảng thời gian rảnh

2.2.1. Thuật toán

Khoảng rảnh là phần bù của hợp các khoảng bận trong khung giờ đang xét. Thuật toán gồm ba bước:

1. Cắt các khoảng bận theo khung giờ của ngày, bỏ qua việc đã hủy. Nếu xét ngày hôm nay thì bỏ phần thời gian đã trôi qua.
2. Sắp xếp các khoảng theo điểm đầu rồi gộp các khoảng chồng nhau hoặc nối tiếp nhau.
3. Duyệt các khoảng đã gộp, lấy phần trống giữa chúng, giữ lại các phần đủ dài.

```
def merge_intervals(intervals: List[Tuple[datetime, datetime]]) -> List[Tuple[datetime,
datetime]]:
    if not intervals:
        return []

    # Sắp xếp theo thời điểm bắt đầu
    sorted_intervals = sorted(intervals, key=lambda x: x[0])
    merged = [sorted_intervals[0]]

    for current_start, current_end in sorted_intervals[1:]:
        last_start, last_end = merged[-1]
        if current_start <= last_end:
            # Có giao thoa hoặc nối tiếp => mở rộng mốc kết thúc
            merged[-1] = (last_start, max(last_end, current_end))
        else:
            merged.append((current_start, current_end))

    return merged
```

Hình 2 minh họa ba bước trên với ngày đầu của dữ liệu mẫu, khi cần một khoảng 60 phút. Hai khoảng 30 phút (nét đứt) bị loại vì không đủ dài.



Hình 2. Các bước tìm khoảng rảnh trong khung 8h đến 21h

2.2.2. Tính đúng đắn

Trong khi gộp, ta duy trì bất biến: các khoảng trong kết quả rời nhau, sắp tăng dần và có hợp bằng hợp các khoảng đã xét. Khi xét khoảng mới, vì các khoảng đã sắp theo điểm đầu nên nó chỉ có thể chạm khoảng cuối của kết quả. Nếu chạm thì nới khoảng cuối, nếu không thì thêm khoảng mới; bất biến vẫn đúng. Khi đã có dãy khoảng bận rời nhau và tăng dần, phần bù chính là các khe giữa hai khoảng liên tiếp, cùng với hai khe ở hai đầu khung giờ.

2.2.3. Chọn khe theo chiến lược

- **Sớm nhất (Earliest):** chọn khe đầu tiên đủ dài.
- **Vừa khít nhất (Best-Fit):** chọn khe có phần dư nhỏ nhất, nhờ đó giữ lại các khoảng trống dài cho việc lớn về sau.

Với dữ liệu mẫu, khi cần 60 phút, chiến lược sớm nhất chọn 12:00–13:00, còn vừa khít nhất chọn 16:30–17:30 (phần dư bằng 0). Best-Fit là một quy tắc kinh nghiệm: nó tốt cho từng lần chọn nhưng không bảo đảm tối ưu cho cả chuỗi yêu cầu về sau, nên chương trình để người dùng tự chọn chiến lược.

2.2.4. Độ phức tạp

Lọc N việc tốn O(N), gộp M khoảng trong ngày tốn O(M log M) do sắp xếp, lấy phần bù tốn O(M), chọn khe tốn O(M). Vì $M \leq N$ nên thời gian là O(N log N), bộ nhớ O(N).

2.3. Sắp xếp thứ tự thực hiện bằng thuật toán Kahn

2.3.1. Thuật toán

Thuật toán Kahn [7] dựa trên nhận xét: việc không còn phải chờ việc nào (bậc vào bằng 0) thì có thể làm ngay. Lấy dần các việc như vậy ra, mỗi lần giảm bậc vào của những việc đang chờ nó.

```
def topological_sort(tasks: List[Task]) -> Tuple[List[Task], List[str]]:
    task_map = {t.id: t for t in tasks}
    in_degree: Dict[str, int] = {t.id: 0 for t in tasks}
    adj_list: Dict[str, List[str]] = {t.id: [] for t in tasks}

    for task in tasks:
        for dep_id in task.dependencies:
            if dep_id in task_map:
                adj_list[dep_id].append(task.id)
                in_degree[task.id] += 1

    # Hàng đợi các đỉnh có bán bậc vào (in_degree) = 0
    queue = deque(tid for tid, deg in in_degree.items() if deg == 0)
    sorted_task_ids = []

    while queue:
        current_id = queue.popleft()
        sorted_task_ids.append(current_id)

        for neighbor_id in adj_list[current_id]:
            in_degree[neighbor_id] -= 1
            if in_degree[neighbor_id] == 0:
                queue.append(neighbor_id)

    if len(sorted_task_ids) != len(tasks):
        # Số đỉnh duyệt được < V => tồn tại chu trình
        unresolved = [t.title for t in tasks if in_degree[t.id] > 0]
        raise ValueError(f"Phát hiện chu trình phụ thuộc (Circular Dependency) giữa các công
việc: {', '.join(unresolved)}")
```

Sau khi có thứ tự, chương trình kiểm tra thêm mỗi cạnh (u, v) xem việc v có bắt đầu trước khi u kết thúc không để đưa ra lưu ý về thời gian.

2.3.2. Tính đúng đắn

- **Thứ tự hợp lệ:** một đỉnh v chỉ vào hàng đợi khi bậc vào giảm về 0, tức là mọi đỉnh u có cạnh tới v đều đã được xuất ra trước.
- **Phát hiện chu trình:** nếu có chu trình thì không đỉnh nào trên chu trình có thể về bậc 0, vì mỗi đỉnh còn chờ đỉnh liền trước trên chu trình, nên số đỉnh xuất ra nhỏ hơn V. Ngược lại, nếu thuật toán dừng khi còn đỉnh chưa xuất, mỗi đỉnh còn lại đều có một đỉnh chờ nằm trong phần còn lại; đi ngược theo các đỉnh đó, do hữu hạn nên phải lặp lại một đỉnh, tức có chu trình.

2.3.3. Độ phức tạp và vai trò của cấu trúc hàng đợi

Mỗi đỉnh vào và ra hàng đợi đúng một lần, mỗi cạnh được xét đúng một lần, nên thời gian là O(V + E), bộ nhớ O(V + E). Kết quả này **chỉ đúng khi lấy phần tử**

đầu hàng đợi trong O(1). Với `list` của Python, `pop(0)` phải dịch toàn bộ phần tử nên tốn O(V) [6], làm thuật toán xấu đi thành $O(V^2)$ trong trường hợp nhiều đỉnh cùng ở hàng đợi. Vì vậy chương trình dùng `collections.deque`; mức chênh lệch được đo ở mục 4.5.4.

2.3.4. Ngăn chu trình ngay khi nhập

Hàm `validate_dependencies` kiểm tra trước khi lưu: tự phụ thuộc, mã không tồn tại, và chu trình bằng cách duyệt theo chiều sâu từ việc đang sửa theo các cạnh tiên quyết, nếu quay lại chính nó thì có chu trình. Vì đồ thị đã lưu luôn là DAG, chu trình mới chỉ có thể đi qua việc đang sửa, nên kiểm tra như vậy là đủ. Thời gian O(V + E). Dữ liệu nhập từ tệp JSON cũng được kiểm tra toàn bộ bằng thuật toán Kahn trước khi ghi.

2.4. Lập lịch khoảng có trọng số bằng quy hoạch động

2.4.1. Phát biểu bài toán

Khi có các việc trùng giờ không thể làm cùng lúc, cần chọn tập con S các việc đôi một không giao nhau sao cho tổng trọng số lớn nhất:

$$\max \sum_{i \in S} w_i \ , \qquad với\ mọi\ i \neq j\ thuộc\ S\colon [s_i, e_i) \cap [s_j, e_j) = \emptyset$$

2.4.2. Hàm trọng số

Trọng số thể hiện mức ưu tiên của người dùng, được chương trình tính bằng công thức:

$$w_i = 10 \cdot 3^{p_i - 1} + \max(1, \frac{120}{\max(30, d_i)})$$

Phần đầu lần lượt bằng 10, 30, 90, 270 cho bốn mức ưu tiên. Dùng cấp số nhân để một việc ưu tiên cao không dễ bị nhiều việc thấp hơn lấn át: việc Khẩn cấp (từ 271 điểm) thắng hai việc Cao (tối đa 2 × 94 = 188 điểm). Phần sau nằm trong khoảng từ 1 đến 4 và chỉ có tác dụng nhỉnh hơn cho việc ngắn khi cùng mức ưu tiên. Đây là lựa chọn thiết kế mang tính chủ quan; nó quyết định "thế nào là tốt", còn thuật toán bảo đảm tìm đúng nghiệm tốt nhất theo định nghĩa đó.

2.4.3. Công thức truy hồi

Sắp xếp các việc theo giờ kết thúc tăng dần $e_1 \le e_2 \le \ldots \le e_n$. Gọi p(j) là số việc có $e_i \le s_j$, đây cũng là chỉ số lớn nhất của việc kết thúc trước khi j bắt đầu (bằng 0 nếu không có). Gọi OPT(j) là tổng trọng số lớn nhất khi chỉ xét j việc đầu tiên:

$$OPT(j) = \max(w_j + OPT(p(j)), \quad OPT(j-1)),\ OPT(0) = 0$$

Chứng minh. Xét một nghiệm tối ưu của j việc đầu tiên, chỉ có hai khả năng. Nếu nghiệm không chứa việc j thì nó là nghiệm của j − 1 việc đầu, giá trị OPT(j − 1). Nếu nghiệm chứa việc j thì mọi việc còn lại phải không giao với j. Vì các việc đã sắp theo giờ kết thúc nên một việc i < j không giao với j khi và chỉ khi $e_i \le s_j$, tức là $i \le p(j)$. Phần còn lại phải là nghiệm tối ưu của p(j) việc đầu, vì nếu có nghiệm tốt hơn thì thay vào sẽ được nghiệm tốt hơn nghiệm tối ưu, vô lý. Lấy giá trị lớn hơn của hai khả năng ta được công thức. Quy nạp theo j cho thấy OPT(n) là giá trị tối ưu.

2.4.4. Cài đặt

Vì mảng `end_times` đã tăng dần, `bisect_right(end_times, s_j)` trả về đúng số phần tử không lớn hơn s_j, tức là p(j). Do $e_j > s_j$ nên p(j) luôn nhỏ hơn j. Sau khi tính xong bảng, truy vết từ cuối về đầu để lấy các việc được chọn.

```
def weighted_interval_scheduling(tasks: List[Task]) -> Tuple[float, List[Task]]:
    if not tasks:
        return 0.0, []

    valid_tasks = [t for t in tasks if t.is_active]
    if not valid_tasks:
        return 0.0, []

    compute_weight = SchedulerEngine.task_weight

    # 1. Sắp xếp các công việc theo thời gian kết thúc tăng dần
    sorted_tasks = sorted(valid_tasks, key=lambda t: t.end_time)
    n = len(sorted_tasks)

    end_times = [t.end_time for t in sorted_tasks]

    # 2. Tiền xử lý tìm p(j) bằng Binary Search (bisect_right)
    p = [0] * (n + 1)
    for j in range(1, n + 1):
        curr_task = sorted_tasks[j - 1]
        # Tìm công việc i có end_time <= curr_task.start_time
        idx = bisect.bisect_right(end_times, curr_task.start_time)
        p[j] = idx

    # 3. Mảng lưu giá trị tối ưu OPT[0..n]
    dp = [0.0] * (n + 1)
    for j in range(1, n + 1):
        weight_j = compute_weight(sorted_tasks[j - 1])
        take = weight_j + dp[p[j]]
        dont_take = dp[j - 1]
        dp[j] = max(take, dont_take)

    # 4. Truy vết tìm tập công việc được chọn
    selected_tasks: List[Task] = []
    curr = n
    while curr > 0:
        weight_curr = compute_weight(sorted_tasks[curr - 1])
        if weight_curr + dp[p[curr]] >= dp[curr - 1]:
            selected_tasks.append(sorted_tasks[curr - 1])
            curr = p[curr]
```

```
        else:
            curr -= 1

    selected_tasks.reverse()
    return dp[n], selected_tasks
```

2.4.5. Độ phức tạp

Sắp xếp O(N log N), tính p(j) gồm N lần tìm kiếm nhị phân O(N log N), điền bảng O(N), truy vết O(N). Vậy thời gian O(N log N), bộ nhớ O(N). So với vét cạn 2^N tập con, đây là khác biệt rất lớn, được đo ở mục 4.5.2.

2.4.6. Ví dụ tính bảng trên dữ liệu mẫu

Dữ liệu mẫu còn 9 việc còn hiệu lực (T01, T02 đã hoàn thành). Bảng 3 là bảng do chương trình tính. Tại j = 2, việc T04 trùng với T03 nên không chọn T04 (31,33 < 271,33). Giá trị tối ưu OPT(9) = 1.050,33.

Bảng 3. Bảng quy hoạch động trên dữ liệu mẫu

j	Việc	w_j	p(j)	Chọn: w_j + OPT(p(j))	Bỏ: OPT(j − 1)	OPT(j)
1	T03	271,33	0	271,33	0,00	271,33
2	T04	31,33	0	31,33	271,33	271,33
3	T05	12,00	2	283,33	271,33	283,33
4	T06	271,00	3	554,33	283,33	554,33
5	T07	91,00	4	645,33	554,33	645,33
6	T08	32,00	5	677,33	645,33	677,33
7	T09	91,00	6	768,33	677,33	768,33
8	T10	271,00	7	1.039,33	768,33	1.039,33
9	T11	11,00	8	1.050,33	1.039,33	1.050,33

Truy vết từ j = 9 cho tập được chọn gồm T03, T05, T06, T07, T08, T09, T10, T11; chỉ T04 bị loại.

2.4.7. Vì sao không dùng thuật toán tham lam

Hai cách tham lam tự nhiên đều không bảo đảm tối ưu. Chọn việc kết thúc sớm nhất tối đa hóa *số việc* nhưng bỏ qua trọng số. Chọn việc trọng số cao nhất trước cũng sai, ví dụ: một việc Khẩn cấp dài 3 giờ (271 điểm) trùng với ba việc Cao dài 1 giờ nối tiếp nhau (mỗi việc 92 điểm, tổng 276). Tham lam chọn việc Khẩn cấp được 271 điểm, còn quy hoạch động chọn ba việc Cao được 276 điểm. Ví dụ này cũng cho thấy hàm trọng số hiện tại đánh đổi: ba việc Cao có thể thắng một việc Khẩn cấp.

2.5. Gỡ trùng lịch tự động

Quy hoạch động mới cho biết nên giữ việc nào, người dùng còn cần biết dời việc còn lại đi đâu. Hàm `plan_conflict_resolution` kết hợp các thuật toán trên để lập phương án:

1. Chạy quy hoạch động, các việc được chọn giữ nguyên giờ và được coi là bận.
2. Sắp các việc còn lại theo ưu tiên giảm dần, cùng ưu tiên thì việc bắt đầu sớm hơn đứng trước.
3. Với mỗi việc, tính mốc sớm nhất có thể: không trước giờ cũ, không trước hiện tại và không trước khi các việc tiên quyết kết thúc.
4. Tìm trong tối đa 14 ngày, trong khung 8h đến 21h, khe rảnh sớm nhất đủ dài, đặt việc vào đó và coi khe đó là bận.
5. Nếu dời việc làm một việc đang phụ thuộc vào nó bắt đầu quá sớm thì đưa ra lưu ý. Phương án chỉ được áp dụng sau khi người dùng xem trước và đồng ý.

Với H là số ngày tìm, D là số việc phải dời, mỗi việc có thể tìm khe trên H ngày, mỗi lần O(N log N), nên thời gian là O(D·H·N log N) (cộng thêm O(N log N) của quy hoạch động). Đây là **thuật toán tham lam**, không bảo đảm phương án tối ưu về tổng thời gian dời, và cũng không tự dời tiếp các việc phụ thuộc. Điều bảo đảm được (và đã kiểm thử ở mục 4.5.5) là sau khi áp dụng, các việc còn hiệu lực không còn trùng giờ, miễn là tìm được chỗ cho tất cả các việc phải dời.

2.6. Phân làn khi vẽ lịch tuần

Trên lịch tuần, các việc trùng giờ phải được vẽ cạnh nhau, không chồng lên nhau. Đây là bài toán phân hoạch khoảng: gán mỗi việc vào một làn sao cho các việc cùng làn không giao nhau, dùng ít làn nhất [2]. Thuật toán duyệt theo giờ bắt đầu và dùng min-heap chứa giờ kết thúc của các làn: nếu làn rảnh sớm nhất đã rảnh thì dùng lại, nếu không thì mở làn mới.

Tính tối ưu. Thuật toán chỉ mở làn thứ d khi cả d − 1 làn đều đang bận tại giờ bắt đầu của việc đang xét, tức là có d việc cùng diễn ra tại thời điểm đó. Mọi cách xếp đều cần ít nhất d làn, nên số làn dùng là ít nhất. Thời gian O(N log N) do sắp xếp và N thao tác trên heap, bộ nhớ O(N).

2.7. Tổng hợp độ phức tạp

Bảng 4. Tổng hợp độ phức tạp của các thuật toán

Thuật toán	Thời gian	Bộ nhớ	Ghi chú
Phát hiện xung đột toàn lịch	O(N log N + K)	O(N + K)	Xấu nhất $O(N^2)$ khi $K \sim N^2$
Kiểm tra trùng khi nhập một việc	O(N)	O(K)	K là số việc trùng với việc đó
Gộp khoảng và tìm khe rảnh	O(N log N)	O(N)	
Sắp xếp topo Kahn	O(V + E)	O(V + E)	Cần hàng đợi lấy đầu O(1)
Kiểm tra phụ thuộc trước khi lưu	O(V + E)	O(V)	Duyệt theo chiều sâu
Quy hoạch động WIS	O(N log N)	O(N)	Nghiệm tối ưu
Gỡ trùng lịch	O(D·H·N log N)	O(N)	Tham lam, không bảo đảm tối ưu
Phân làn lịch tuần	O(N log N)	O(N)	Số làn ít nhất
Phân loại Eisenhower	O(N)	O(N)	

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG CHƯƠNG TRÌNH

3.1. Ngôn ngữ và công cụ

– **Python 3:** cú pháp rõ ràng, có sẵn các cấu trúc dữ liệu cần thiết (`bisect`, `heapq`, `deque`), phù hợp để tập trung vào thuật toán.

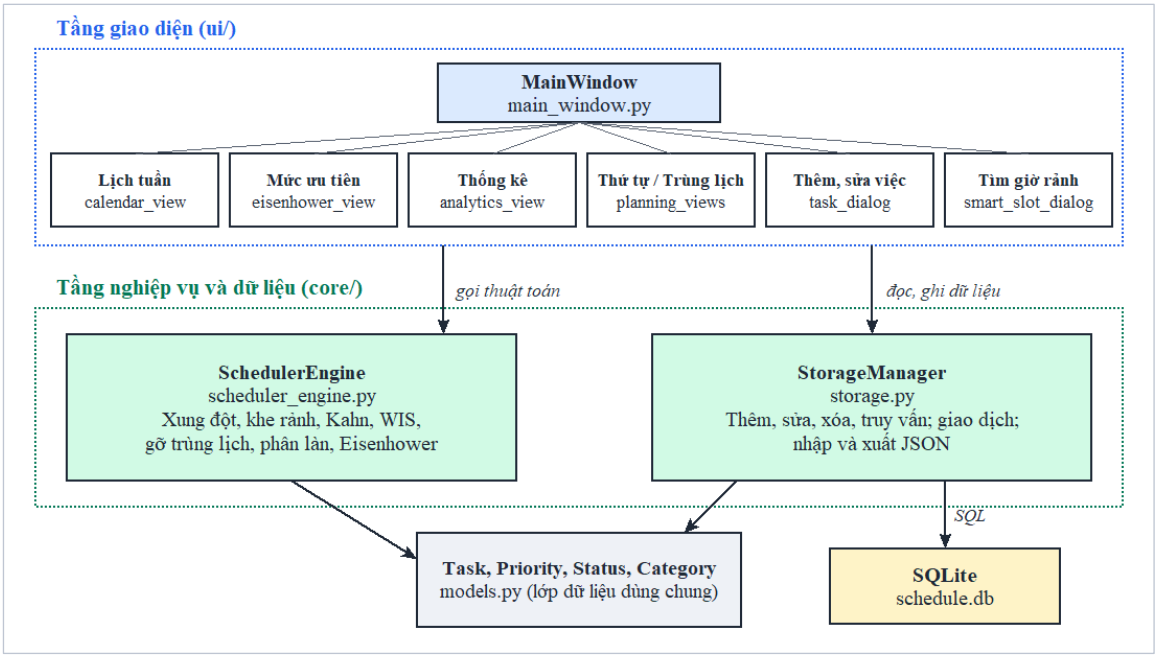
– **Tkinter:** thư viện giao diện có sẵn của Python. Lịch tuần và các biểu đồ được tự vẽ bằng `Canvas` nên không cần thư viện ngoài.

– **SQLite:** cơ sở dữ liệu nhúng trong một tệp, không cần cài máy chủ, có giao dịch và khóa ngoại.

– **unittest:** kiểm thử tự động cho cả thuật toán, lưu trữ và giao diện.

3.2. Kiến trúc chương trình

Chương trình chia thành hai tầng (Hình 3). Tầng `core` không phụ thuộc giao diện nên có thể kiểm thử và đo hiệu năng riêng. Tầng `ui` chỉ nhận thao tác, gọi tầng `core` và hiển thị kết quả.



Hình 3. Kiến trúc chương trình

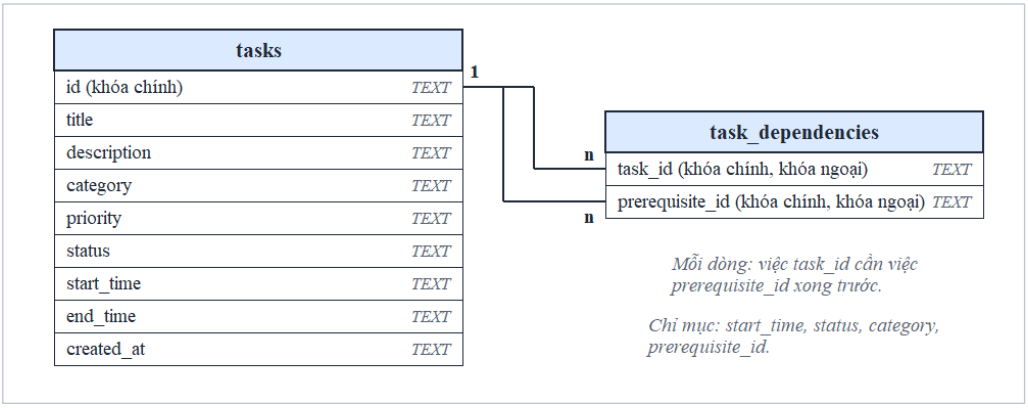
3.3. Thiết kế lớp dữ liệu

Lớp `Task` đại diện cho một công việc. Hàm khởi tạo từ chối ngay trường hợp giờ kết thúc không sau giờ bắt đầu, nên trong chương trình không tồn tại đối tượng có thời gian sai. Thuộc tính `is_active` gom quy tắc "còn hiệu lực" về một chỗ để mọi thuật toán dùng chung. `Priority`, `Status`, `Category` là các kiểu liệt kê (`Enum`) chứa cả nhãn tiếng Việt và màu hiển thị.

```
def is_overlapping_with(self, other: 'Task') -> bool:
    if self.id == other.id:
        return False
    return max(self.start_time, other.start_time) < min(self.end_time, other.end_time)
```

3.4. Thiết kế cơ sở dữ liệu

Quan hệ phụ thuộc là quan hệ nhiều - nhiều giữa các công việc nên được tách thành bảng riêng (Hình 4).



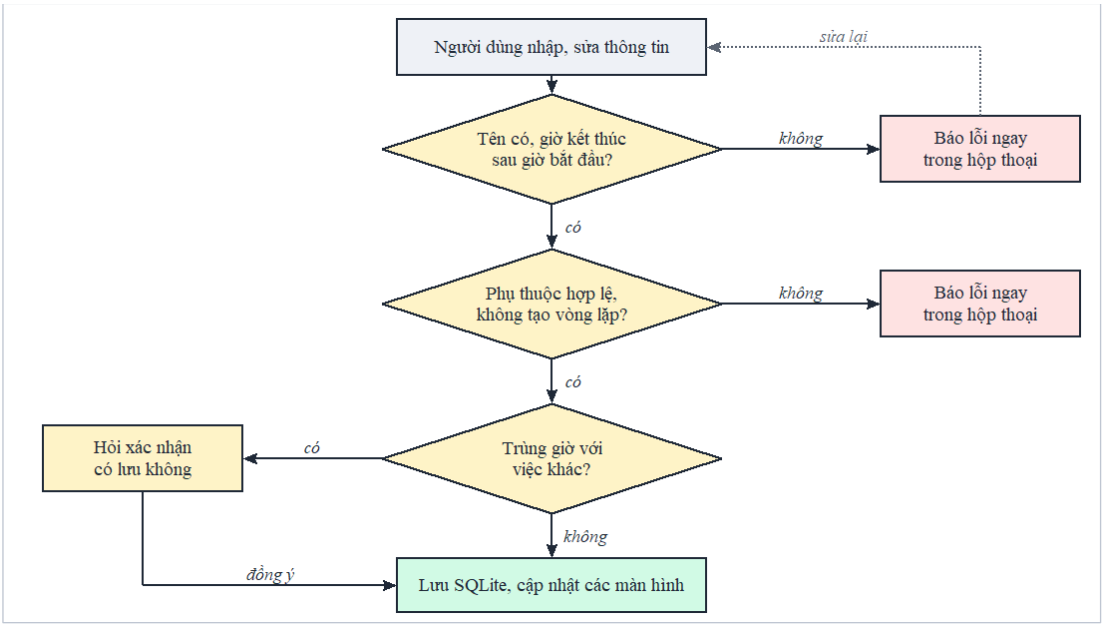
Hình 4. Lược đồ cơ sở dữ liệu

Một số quyết định thiết kế đáng chú ý:

- **Bật khóa ngoại trên mỗi kết nối** (`PRAGMA foreign_keys = ON`), vì SQLite mặc định tắt [8]. Nhờ đó không thể lưu phụ thuộc trỏ tới việc không tồn tại.
- **Dùng UPSERT thay cho INSERT OR REPLACE.** REPLACE xóa dòng cũ rồi thêm dòng mới, khi khóa ngoại đang bật sẽ kéo theo xóa phụ thuộc của các việc khác trỏ tới việc đó. Có kiểm thử riêng cho tình huống này.
- **Ghi nhiều việc trong một giao dịch:** nhập JSON và áp dụng phương án dời việc, lỗi ở đâu cũng không để lại dữ liệu dở dang.

3.5. Xử lý khi lưu công việc

Các lỗi được chặn trước khi tới cơ sở dữ liệu (Hình 5). Riêng trùng giờ không phải lỗi, vì người dùng có thể cố ý xếp trùng, nên chương trình chỉ cảnh báo và hỏi xác nhận.



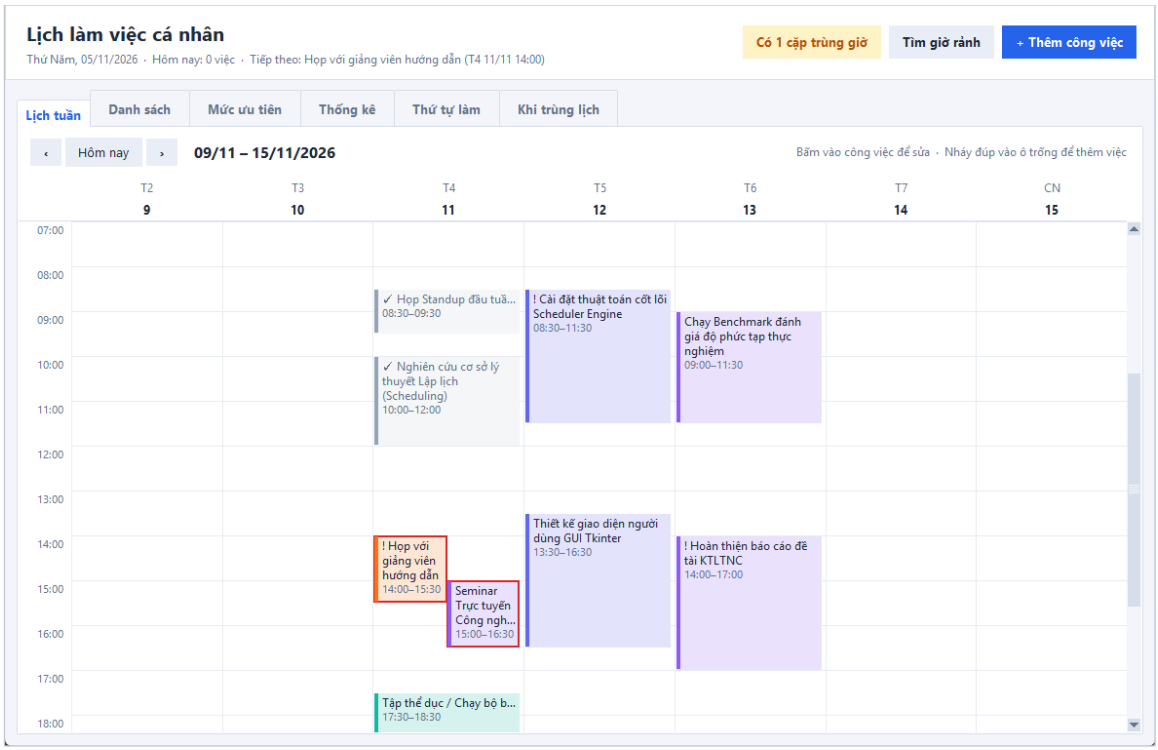
Hình 5. Luồng kiểm tra khi lưu công việc

3.6. Thiết kế giao diện

Giao diện được thiết kế theo các nguyên tắc sau:

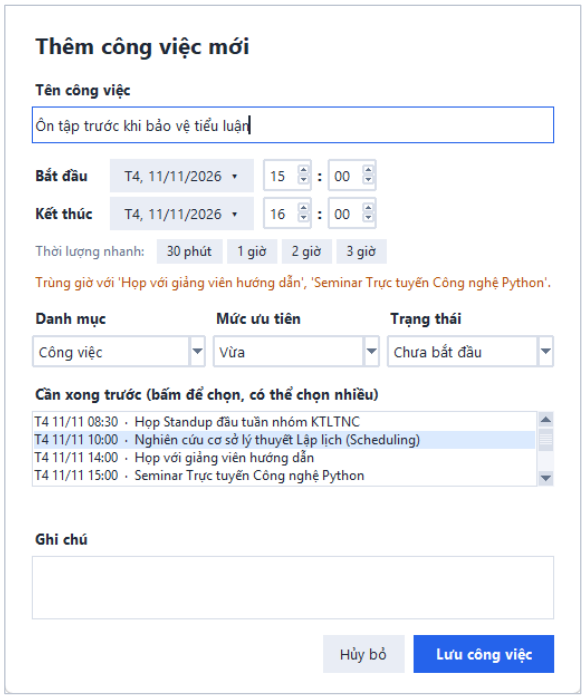
- **Nói bằng ngôn ngữ người dùng:** tab tên là "Thứ tự làm", "Khi trùng lịch"; tên thuật toán chỉ nằm ở dòng giải thích.
- **Không bắt người dùng nhớ định dạng:** chọn ngày trên lịch tháng, chọn việc tiên quyết theo tên thay vì gõ mã.
- **Phản hồi ngay:** thông báo trùng giờ, vòng lặp phụ thuộc đổi ngay khi đang nhập, không cần bấm kiểm tra.
- **Không làm gián đoạn:** thao tác thành công chỉ hiện thông báo ngắn tự ẩn; chỉ hỏi xác nhận với thao tác xóa hoặc có rủi ro.
- **Chương trình đề xuất, người dùng quyết định:** phương án dời việc luôn được xem trước khi áp dụng.

Màn hình chính là lịch tuần (Hình 6). Các việc trùng giờ được xếp cạnh nhau, viền đỏ, dấu "!" chỉ việc Khẩn cấp, dấu "✓" chỉ việc đã hoàn thành. Dòng đầu cửa sổ tóm tắt số việc hôm nay và việc tiếp theo; nút màu vàng cho biết có bao nhiêu cặp trùng giờ.

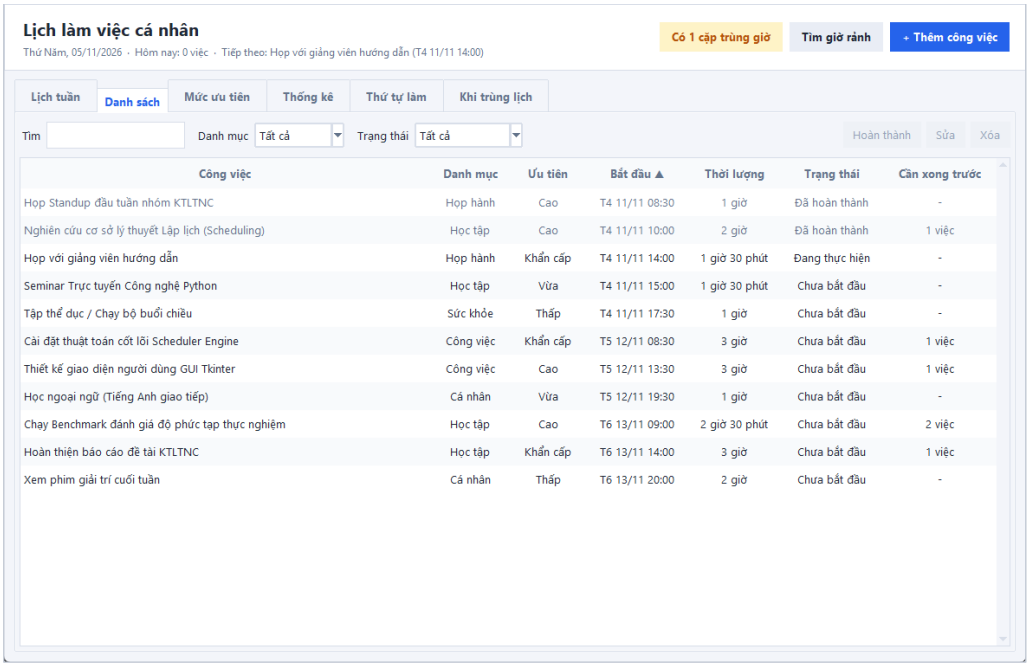


Hình 6. Màn hình lịch tuần

Hình 7 là hộp thoại thêm công việc: ngày chọn bằng lịch tháng, có nút thời lượng nhanh, dòng trạng thái báo ngay việc đang nhập trùng giờ với việc nào. Hình 8 là màn hình danh sách với tìm kiếm tức thì, bộ lọc và sắp xếp theo cột.



Hình 7. Hộp thoại thêm công việc



Hình 8. Màn hình danh sách công việc

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Môi trường và phương pháp

Bảng 5. Môi trường thực nghiệm

Thành phần	Thông số
Bộ xử lý	Intel Core i5-1035G1, xung cơ bản 1,00 GHz
Bộ nhớ	Khoảng 20 GB RAM
Hệ điều hành	Windows 11 Pro
Ngôn ngữ	Python 3.12.12, Tk 8.6

Chương trình được đánh giá theo ba câu hỏi, mỗi câu dùng một phương pháp riêng:

- **Có chạy đúng trên tình huống cụ thể không?** Chạy trên bộ dữ liệu mẫu có chủ đích, tự kiểm tra lại kết quả (mục 4.2, 4.3).
- **Có đúng trên mọi dữ liệu không?** Đối chiếu tự động với nghiệm vét cạn trên dữ liệu ngẫu nhiên (mục 4.4).
- **Có nhanh như lý thuyết không?** Đo thời gian khi nhân đôi quy mô, so sánh với cách làm đơn giản hơn (mục 4.5).

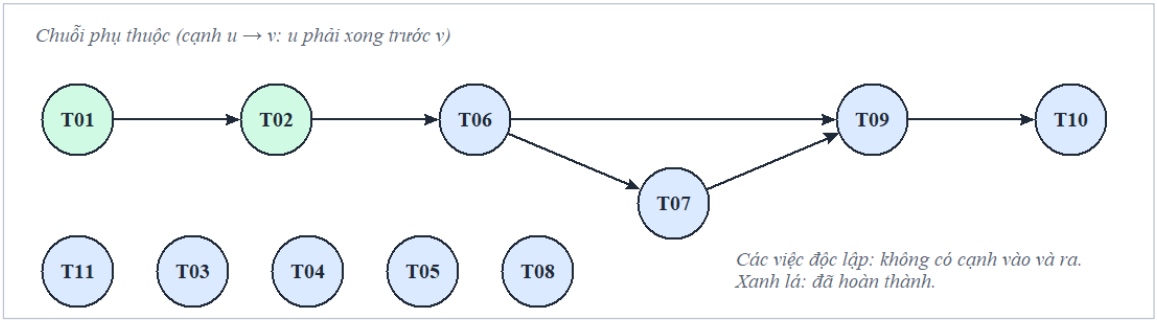
Có thể chạy lại mọi kết quả bằng các tệp `demo_experiments.py`, `experiments.py` và `benchmark.py`. Dữ liệu sinh với seed cố định; trừ thời gian đo, các kết quả là giống nhau giữa các lần chạy.

4.2. Bộ dữ liệu mẫu

Bộ dữ liệu gồm 11 công việc trong 3 ngày (Bảng 6), được thiết kế để có đủ tình huống cần kiểm tra: một cặp trùng giờ có chủ đích (T03 và T04), một chuỗi phụ thuộc nhiều bậc có nhánh (Hình 9), và đủ các mức ưu tiên, trạng thái.

Bảng 6. Bộ dữ liệu mẫu (T01, T02 đã hoàn thành)

Mã	Công việc	Ưu tiên	Bắt đầu	Kết thúc	Cần xong trước
T01	Họp Standup đầu tuần nhóm KTLTNC	Cao	04/11 08:30	09:30	-
T02	Nghiên cứu cơ sở lý thuyết Lập lịch (Scheduling)	Cao	04/11 10:00	12:00	T01
T03	Họp với giảng viên hướng dẫn	Khẩn cấp	04/11 14:00	15:30	-
T04	Seminar Trực tuyến Công nghệ Python	Vừa	04/11 15:00	16:30	-
T05	Tập thể dục / Chạy bộ buổi chiều	Thấp	04/11 17:30	18:30	-
T06	Cài đặt thuật toán cốt lõi Scheduler Engine	Khẩn cấp	05/11 08:30	11:30	T02
T07	Thiết kế giao diện người dùng GUI Tkinter	Cao	05/11 13:30	16:30	T06
T08	Học ngoại ngữ (Tiếng Anh giao tiếp)	Vừa	05/11 19:30	20:30	-
T09	Chạy Benchmark đánh giá độ phức tạp thực nghiệm	Cao	06/11 09:00	11:30	T06, T07
T10	Hoàn thiện báo cáo đề tài KTLTNC	Khẩn cấp	06/11 14:00	17:00	T09
T11	Xem phim giải trí cuối tuần	Thấp	06/11 20:00	22:00	-

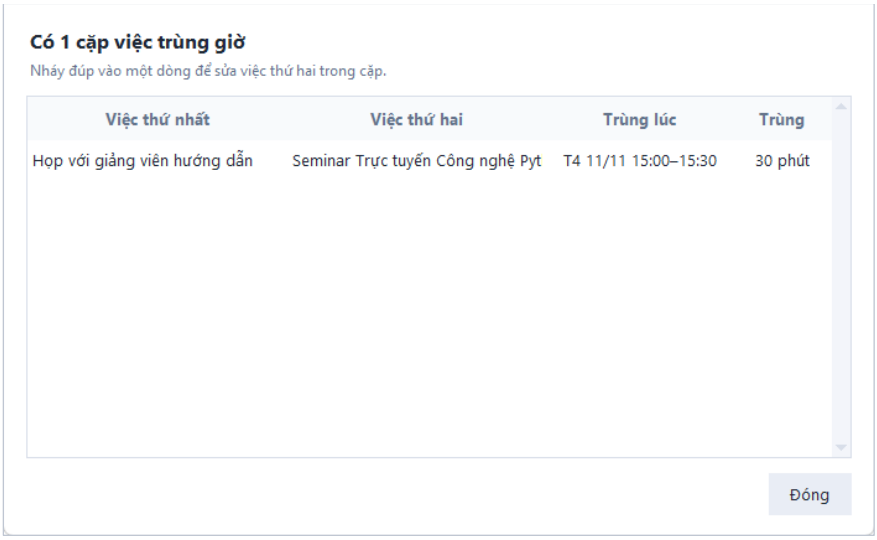


Hình 9. Đồ thị phụ thuộc của bộ dữ liệu mẫu

4.3. Kết quả chạy trên dữ liệu mẫu

4.3.1. Phát hiện trùng giờ

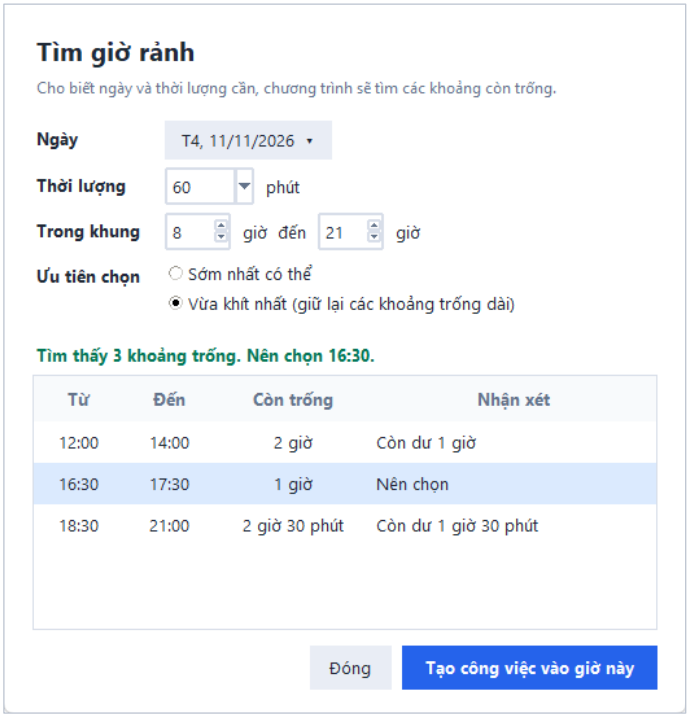
Chương trình phát hiện đúng một cặp là T03 và T04, trùng từ 15:00 đến 15:30, tức 30 phút, khớp với phần tính tay theo công thức ở mục 1.3.1 (Hình 10). Các việc nối tiếp nhau không bị báo nhầm.



Hình 10. Danh sách các cặp việc trùng giờ

4.3.2. Tìm giờ rảnh

Trong ngày 04/11, khung 8h đến 21h, cần 60 phút, chương trình tìm được ba khe: 12:00–14:00 (120 phút), 16:30–17:30 (60 phút), 18:30–21:00 (150 phút), khớp với Hình 2. Khi chọn chiến lược vừa khít nhất, dòng được chọn sẵn là khe 16:30 có phần dư bằng 0 (Hình 11). Bấm "Tạo công việc vào giờ này" sẽ mở hộp thoại với đúng giờ đã chọn.

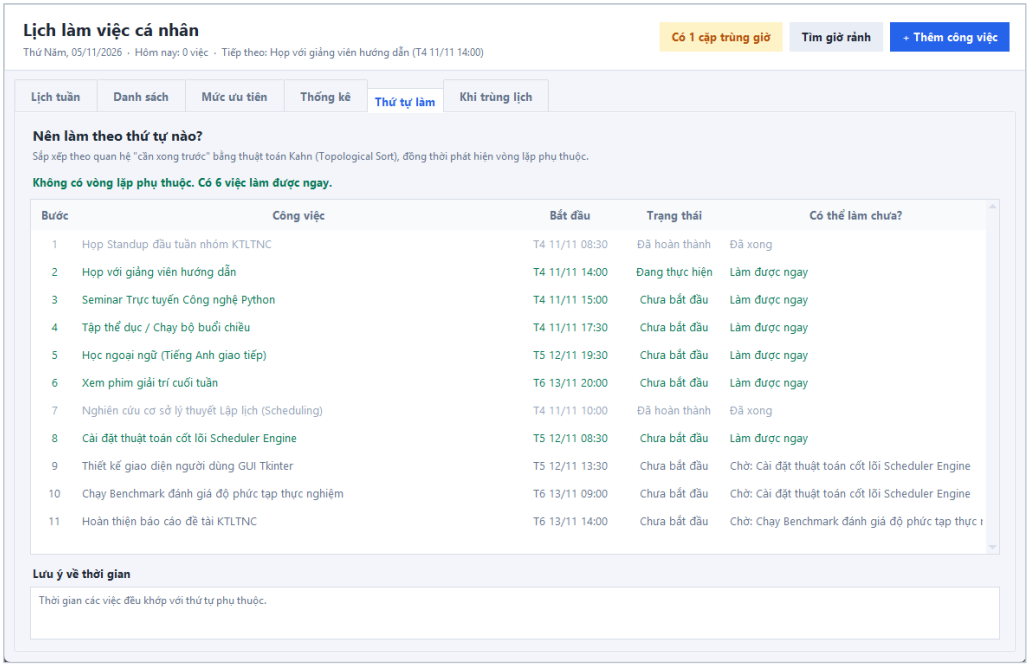


Hình 11. Tìm giờ rảnh theo chiến lược vừa khít nhất

4.3.3. Thứ tự thực hiện

Thuật toán Kahn cho thứ tự: T01 → T03 → T04 → T05 → T08 → T11 → T02 → T06 → T07 → T09 → T10. Kiểm tra trên Hình 9: mọi cạnh đều đi từ việc đứng

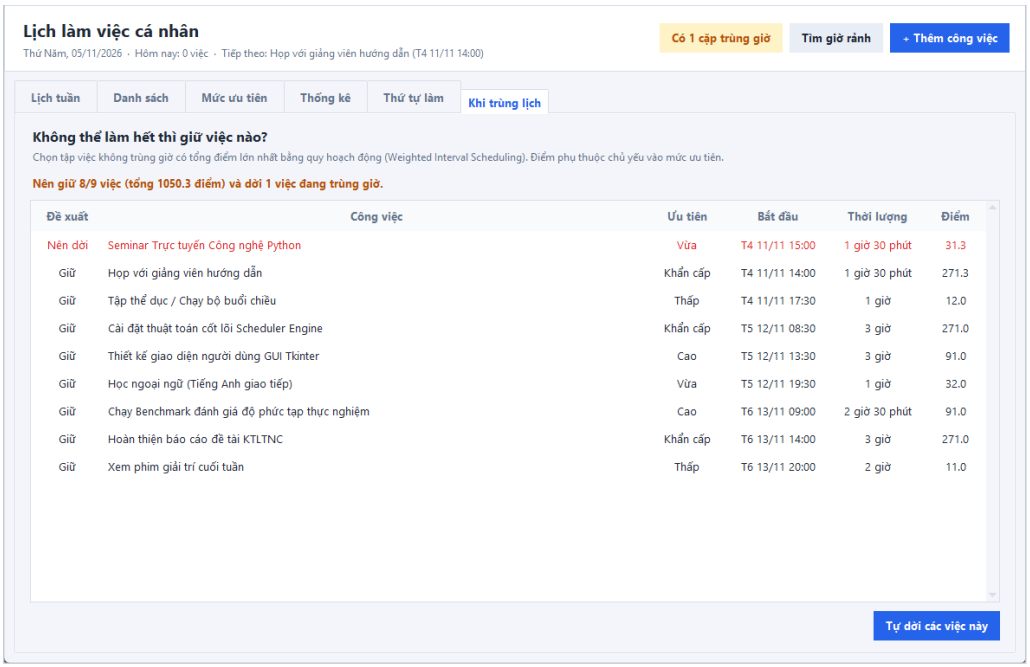
trước tới việc đứng sau, ví dụ T06 đứng trước T07, T07 đứng trước T09. Ngoài ra, khi đưa vào chu trình ba việc, hàm ném ngoại lệ và nêu đúng tên các việc liên quan. Hình 12 là kết quả trên giao diện, có thêm cột cho biết việc nào làm được ngay, việc nào còn đang chờ.



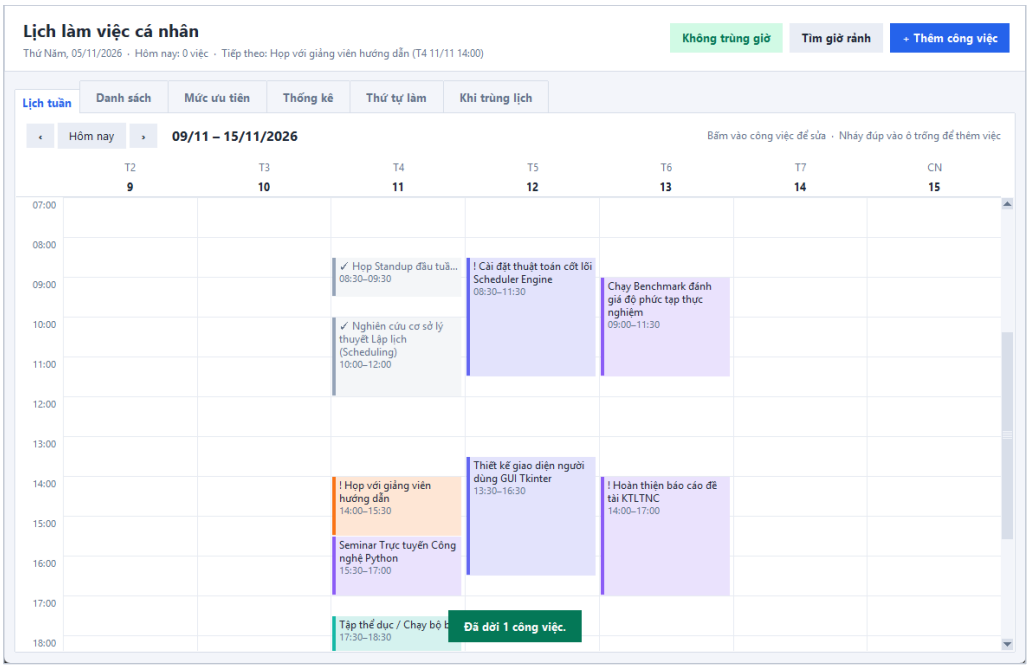
Hình 12. Màn hình thứ tự thực hiện

4.3.4. Tối ưu và gỡ trùng lịch

Quy hoạch động giữ 8/9 việc, tổng 1.050,33 điểm, đề xuất dời T04 (Hình 13). Kết quả hợp lý vì T03 là việc Khẩn cấp (271,33 điểm) còn T04 chỉ ở mức Vừa (31,33 điểm). Khi bấm "Tự dời các việc này", chương trình đề xuất dời T04 sang 15:30, ngay sau khi T03 kết thúc, trước giờ của T05. Sau khi áp dụng, nút cảnh báo chuyển thành "Không trùng giờ" và trên lịch hai việc không còn chồng lên nhau (Hình 14).



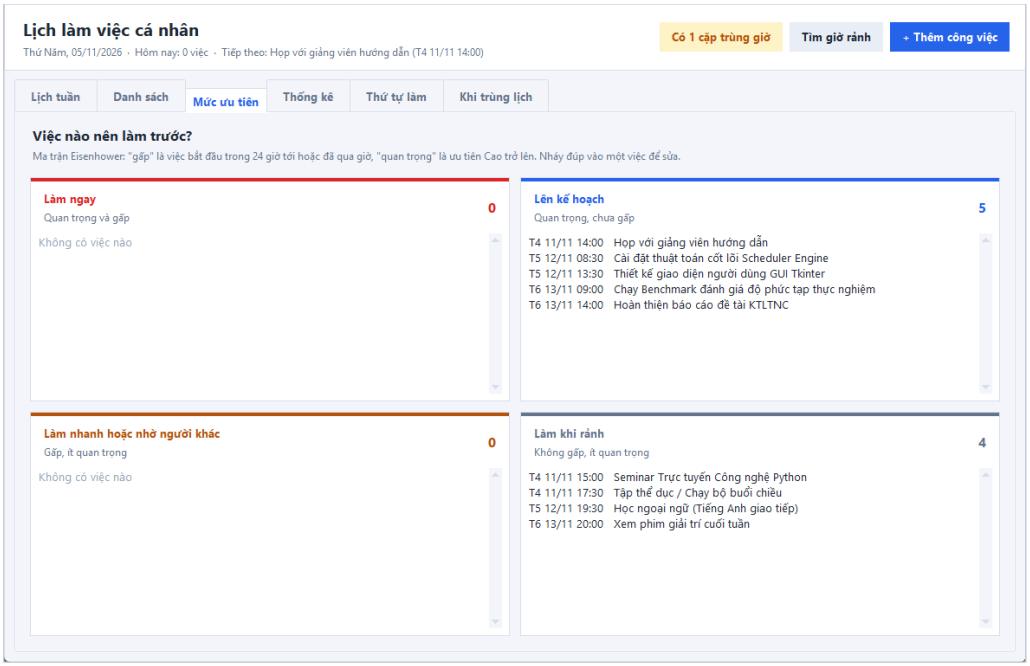
Hình 13. Màn hình đề xuất khi trùng lịch



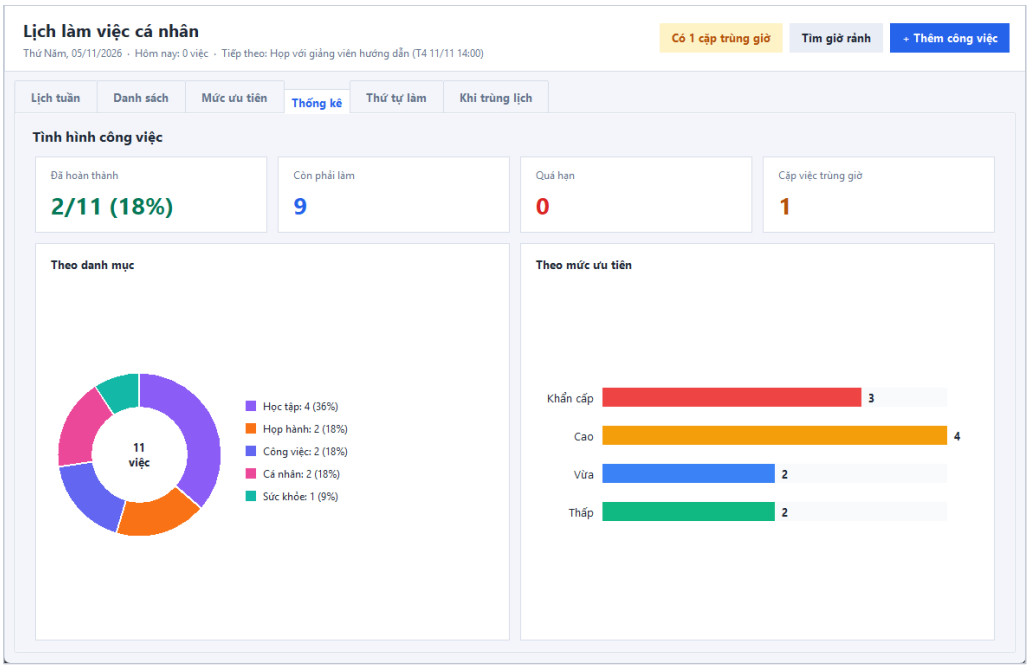
Hình 14. Lịch tuần sau khi tự động dời việc T04

4.3.5. Mức ưu tiên và thống kê

Hình 15 là ma trận Eisenhower, Hình 16 là màn hình thống kê với biểu đồ tự vẽ bằng `Canvas`.



Hình 15. Ma trận Eisenhower



Hình 16. Màn hình thống kê

4.4. Kiểm tra tính đúng đắn

4.4.1. Đối chiếu với nghiệm vét cạn

Vài ví dụ chạy đúng chưa đủ để kết luận thuật toán đúng. Vì vậy em viết thêm các lời giải vét cạn đơn giản (ít khả năng sai) rồi so sánh tự động với thuật toán chính trên 1.500 bộ dữ liệu ngẫu nhiên, mỗi bộ tối đa 11 việc (Bảng 7).

Bảng 7. Kết quả đối chiếu trên 1.500 bộ dữ liệu ngẫu nhiên

Thuật toán	Lời giải đối chiếu	Số lần sai
Phát hiện xung đột (số cặp)	Xét mọi cặp	0
Quy hoạch động (giá trị tối ưu)	Xét mọi tập con	0
Quy hoạch động (truy vết: tập hợp lệ, đúng tổng)	Kiểm tra trực tiếp tập được chọn	0
Tìm khe rảnh	Đánh dấu từng phút trong ngày	0

Không có trường hợp sai nào. Kết quả này là bằng chứng thực nghiệm, bổ trợ chứ không thay thế cho các lập luận ở chương 2.

4.4.2. Kiểm thử tự động

Bộ kiểm thử gồm 62 ca, tất cả đều đạt (Bảng 8). Ngoài các trường hợp thông thường, bộ kiểm thử bao gồm các tình huống dễ sai: việc nối tiếp nhau, chu trình phụ thuộc, tệp JSON hỏng, sửa một việc mà không làm mất phụ thuộc của việc khác. Các ca kiểm thử giao diện chạy trên cửa sổ thật với cơ sở dữ liệu tạm.

Bảng 8. Các nhóm kiểm thử tự động

Nhóm	Nội dung chính	Số ca
Thuật toán và lưu trữ (`test_scheduler.py`)	Mô hình, xung đột, khe rảnh, Kahn, quy hoạch động, gỡ trùng lịch, phân làn, SQLite, JSON	40
Giao diện (`test_ui.py`)	Điền đúng giờ đã chọn, chặn vòng lặp, biểu đồ, bộ lọc, bấm trên lịch, tự dời việc	22
Tổng cộng		**62**

4.5. Đánh giá hiệu năng

4.5.1. Phát hiện xung đột: ảnh hưởng của mật độ lịch

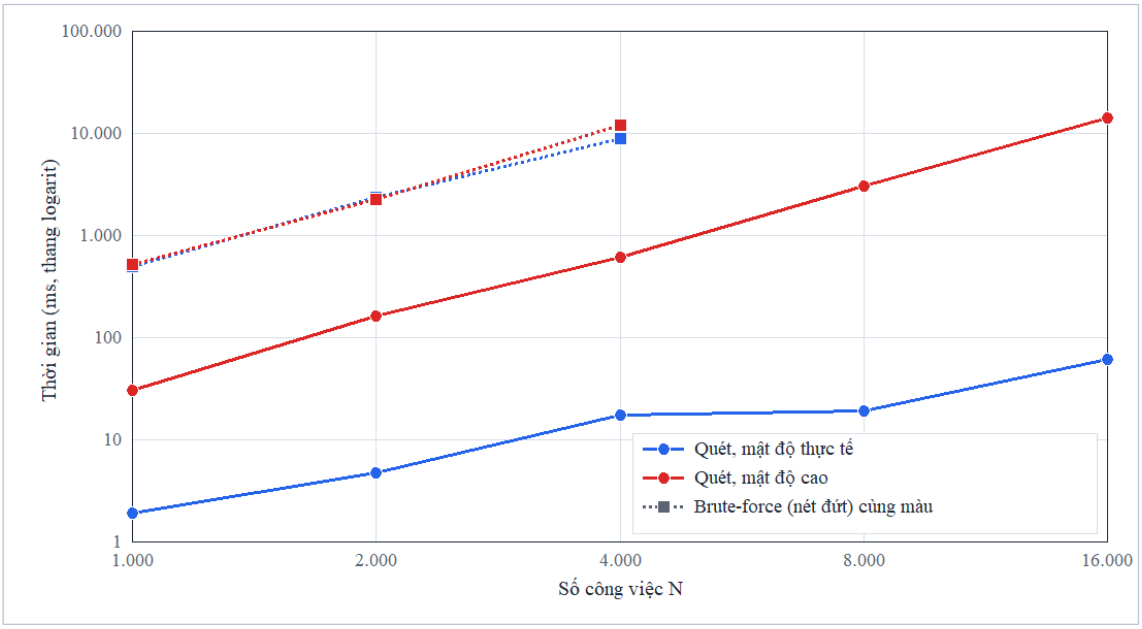
Do độ phức tạp phụ thuộc K, em đo ở hai mật độ: **mật độ thực tế** (khoảng 8 việc/ngày) và **mật độ cao** (mọi việc dồn vào 5 ngày, K tăng theo N^2). Mỗi số đo là trung vị của 5 lần chạy; brute-force đo 1 lần, chỉ tới N = 4.000 vì quá lâu. Để so sánh công bằng, brute-force cũng tạo cùng kết quả như thuật toán chính.

Bảng 9. Phát hiện xung đột ở mật độ thực tế

N	K	Quét (ms)	Tăng	Brute-force (ms)	Nhanh hơn
1.000	559	1,90	-	501,19	264 lần
2.000	1.103	4,75	×2,5	2.346,97	494 lần
4.000	2.204	17,42	×3,7	8.873,20	509 lần
8.000	4.546	19,39	×1,1	không đo	-
16.000	8.972	61,39	×3,2	không đo	-

Bảng 10. Phát hiện xung đột ở mật độ cao

N	K	Quét (ms)	Tăng	Brute-force (ms)	Nhanh hơn
1.000	13.861	30,66	-	516,67	17 lần
2.000	56.781	161,71	×5,3	2.274,59	14 lần
4.000	225.270	612,94	×3,8	11.969,21	20 lần
8.000	895.044	3.052,14	×5,0	không đo	-
16.000	3.582.291	14.137,27	×4,6	không đo	-



Hình 17. Thời gian phát hiện xung đột theo N (hai trục đều là thang logarit)

Nhận xét:

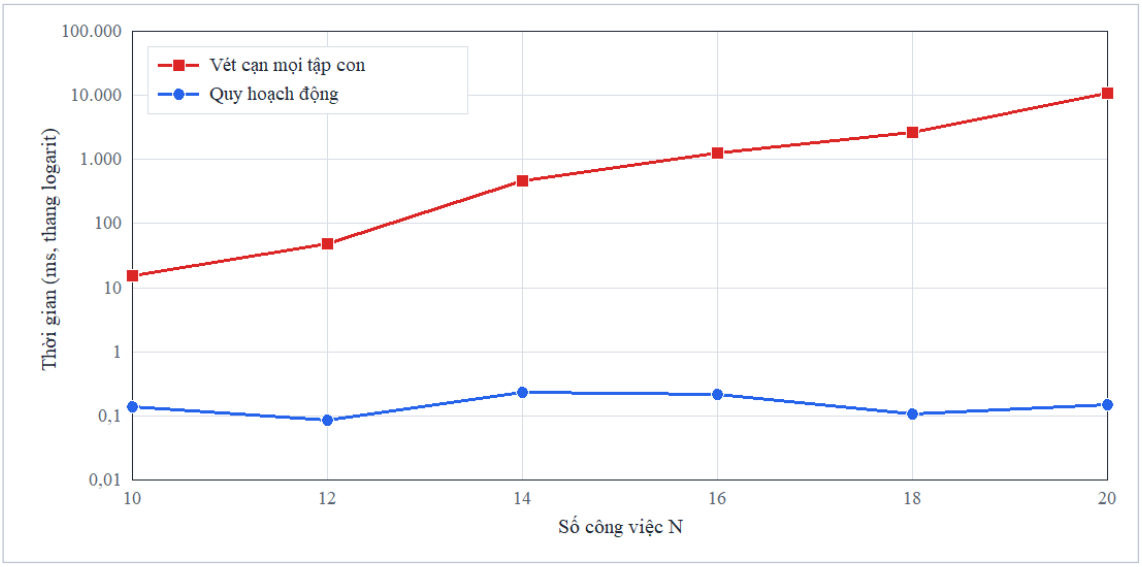
- **Ở mật độ cao,** K tăng khoảng 4 lần khi N gấp đôi và thời gian quét cũng tăng khoảng 4–5 lần. Thuật toán vẫn gần bậc hai, đúng như đánh giá O(N log N + K) khi K ~ N². Nó vẫn nhanh hơn brute-force từ 14 đến 20 lần, nhưng chênh lệch này gần như không tăng theo N, tức chỉ là cải thiện hằng số.
- **Ở mật độ thực tế,** K nhỏ nên thuật toán nhanh hơn brute-force hàng trăm lần và chênh lệch tăng theo N. Với N = 16.000, thời gian chỉ khoảng 61 ms.
- **Điểm chưa khớp hoàn toàn:** khi N tăng 16 lần (từ 1.000 lên 16.000), thời gian ở mật độ thực tế tăng khoảng 32 lần, cao hơn mức khoảng 22 lần mà N log N dự đoán, và các hệ số tăng từng bước dao động mạnh (từ ×1,1 đến ×3,7). Với số đo chỉ vài mili-giây, nhiễu của hệ điều hành và bộ thu gom rác của Python ảnh hưởng đáng kể; em chưa tách riêng được các yếu tố này.

Kết luận rút ra: nói thuật toán "giảm từ $O(N^2)$ xuống $O(N \log N)$" là không chính xác. Phát biểu đúng là thuật toán nhanh theo đầu ra, phát huy hiệu quả khi lịch thưa.

4.5.2. Quy hoạch động so với vét cạn

Bảng 11. Quy hoạch động so với vét cạn mọi tập con

N	Vét cạn (ms)	Quy hoạch động (ms)	Cùng giá trị tối ưu
10	15,33	0,140	Có
12	48,19	0,085	Có
14	457,13	0,230	Có
16	1.253,57	0,219	Có
18	2.685,38	0,107	Có
20	10.950,34	0,150	Có



Hình 18. Thời gian vét cạn và quy hoạch động (trục tung là thang logarit)

Với N = 20, vét cạn mất khoảng 11,0 giây trong khi quy hoạch động chưa tới 1 mili-giây, hai bên cho cùng giá trị tối ưu ở mọi N đã đo. Thời gian vét cạn không tăng đều vì mỗi N dùng một bộ dữ liệu khác nhau và việc kiểm tra tập con được dừng sớm khi gặp cặp trùng, nhưng xu hướng tăng theo hàm mũ là rõ ràng. Số đo của quy hoạch động quá nhỏ nên chủ yếu là nhiễu.

4.5.3. Quy hoạch động so với tham lam

Mỗi N chạy 1.000 bộ dữ liệu ngẫu nhiên trong một ngày, đo phần trăm tổng điểm thiếu so với nghiệm tối ưu của quy hoạch động (Bảng 12).

Bảng 12. Chất lượng nghiệm của thuật toán tham lam so với quy hoạch động

N	Tham lam theo trọng số: thiếu trung bình	Thiếu nhiều nhất	Tỷ lệ đạt tối ưu	Tham lam theo giờ kết thúc: thiếu trung bình	Tỷ lệ đạt tối ưu
10	0,4%	40,6%	94,5%	22,5%	19,4%
20	0,7%	27,5%	85,3%	29,4%	3,8%
50	1,6%	30,0%	63,3%	36,3%	0,1%
100	2,3%	23,7%	43,3%	41,2%	0,0%

Tham lam theo trọng số khá tốt về trung bình (thiếu 0,4% đến 2,3%), đây là kết quả dễ hiểu vì hàm trọng số tăng rất nhanh theo ưu tiên. Tuy nhiên, tỷ lệ đạt tối ưu giảm dần theo N (còn 43,3% với N = 100), và có trường hợp thiếu tới 40,6%. Tham lam theo giờ kết thúc thiếu nhiều hơn hẳn vì bỏ qua trọng số. Vì quy hoạch động có cùng bậc độ phức tạp với tham lam nhưng luôn tối ưu nên đây là lựa chọn hợp lý.

4.5.4. Ảnh hưởng của cấu trúc hàng đợi trong thuật toán Kahn

Đo trên trường hợp xấu nhất là mọi đỉnh đều ở hàng đợi ngay từ đầu (Bảng 13), mỗi cỡ đo một lần.

Bảng 13. Thuật toán Kahn với hai cấu trúc hàng đợi

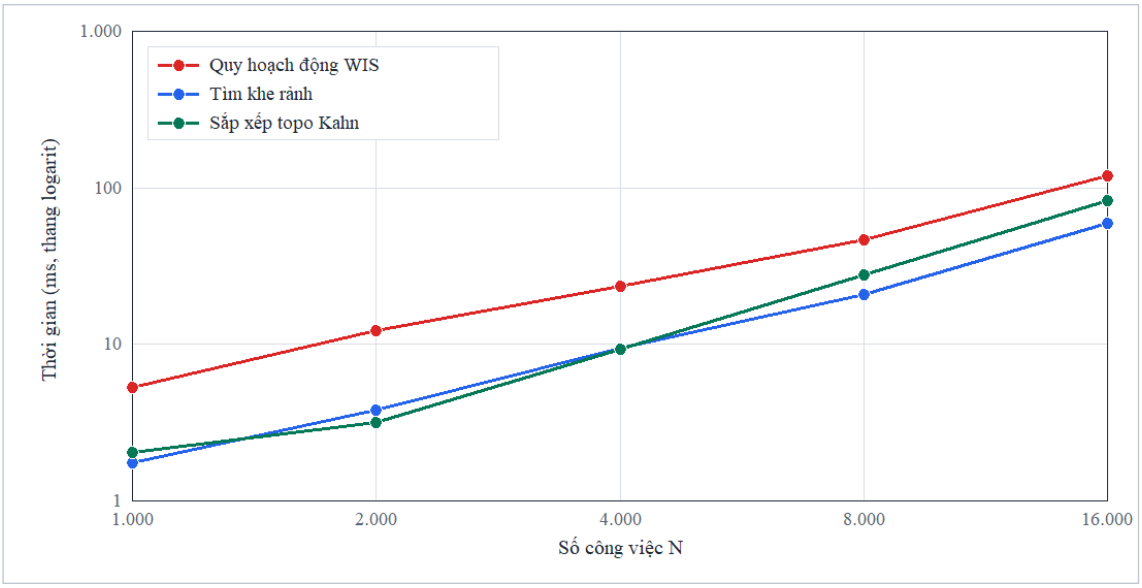
Số đỉnh V	`list.pop(0)` (ms)	`deque.popleft()` (ms)	Chậm hơn
50.000	1.382,9	21,3	65 lần
100.000	16.445,5	218,8	75 lần
200.000	43.393,6	115,8	375 lần
400.000	172.493,5	281,6	612 lần

Khi V tăng 8 lần (từ 50.000 lên 400.000), cách dùng `list` chậm đi khoảng 125 lần, thậm chí vượt mức 64 lần mà $O(V^2)$ dự đoán, trong khi `deque` chỉ chậm đi khoảng 13 lần (tuyến tính lý tưởng là 8 lần; mỗi cỡ chỉ đo một lần nên có nhiễu, ví dụ ở V = 100.000). Một chi tiết cài đặt nhỏ đã làm thay đổi bậc độ phức tạp. Ở quy mô lịch cá nhân, sự khác biệt này không nhận thấy được, nhưng chỉ khi dùng `deque` thì đánh giá $O(V + E)$ mới đúng với mã nguồn.

4.5.5. Các thuật toán còn lại và gỡ trùng lịch

Bảng 14. Thời gian các thuật toán còn lại (trung vị của 5 lần chạy)

N	Tìm khe rảnh (ms)	Quy hoạch động (ms)	Kahn (ms)
1.000	1,76	5,34	2,03
2.000	3,79	12,29	3,18
4.000	9,40	23,39	9,25
8.000	20,81	46,43	27,66
16.000	59,37	118,52	82,48



Hình 19. Thời gian các thuật toán còn lại theo N (hai trục đều là thang logarit)

Khi N tăng 16 lần, quy hoạch động tăng khoảng 22 lần, khớp tốt với dự đoán khoảng 22 lần của N log N. Tìm khe rảnh tăng khoảng 34 lần và Kahn tăng khoảng 41 lần, đều cao hơn lý thuyết (Kahn lý tưởng là 16 lần). Nguyên nhân có thể là bộ nhớ đệm của bộ xử lý kém hiệu quả hơn khi dữ liệu lớn và bộ thu gom rác của Python, em chưa kiểm chứng riêng. Mặc dù vậy, với N = 16.000 cả ba đều dưới 120 ms, thừa đáp ứng cho lịch cá nhân.

Với chức năng gỡ trùng lịch, em chạy 500 bộ dữ liệu ngẫu nhiên, mỗi bộ 5 đến 20 việc dồn vào một ngày (6.204 việc, 9.637 cặp trùng ban đầu). Sau khi áp dụng phương án còn **0 cặp trùng**, không có việc nào không xếp được, 3.609 việc được dời. Thời gian dời trung bình khoảng 17,2 giờ, nhiều nhất 71 giờ. Con số này lớn vì dữ liệu thử cố tình rất dày; em chưa so với phương án tối ưu nên chưa kết luận được phương án tham lam kém tối ưu bao nhiêu.

4.6. Đánh giá chung và hạn chế

Đối chiếu với yêu cầu đề tài, chương trình đã có đủ các chức năng đề ra, các thuật toán cho kết quả đúng trong toàn bộ phép đối chiếu, và đủ nhanh ở quy mô lịch cá nhân. Tuy nhiên, chương trình còn những hạn chế sau:

- **Mô hình dữ liệu:** chưa có hạn chót riêng nên mức gấp tính theo giờ bắt đầu; chưa hỗ trợ việc lặp lại và chưa có nhắc việc.
- **Hàm trọng số mang tính chủ quan:** người dùng chưa chỉnh được; ba việc Cao có thể thắng một việc Khẩn cấp như đã nêu ở mục 2.4.7.
- **Gỡ trùng lịch là tham lam:** không tối ưu tổng thời gian dời, không tự dời tiếp các việc phụ thuộc mà chỉ đưa ra lưu ý.
- **Phép đo hiệu năng:** thực hiện trên một máy, các phép đo rất ngắn có nhiễu, và một số độ tăng cao hơn lý thuyết chưa được giải thích đầy đủ.
- **Giao diện:** chưa được thử nghiệm với người dùng độc lập nên mức độ thân thiện mới là đánh giá của người làm.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đề tài đã xây dựng được chương trình quản lý lịch làm việc cá nhân bằng Python, có giao diện đồ họa và lưu trữ bền vững, đáp ứng các yêu cầu đặt ra về lý thuyết và thực nghiệm:

- **Về lý thuyết:** đã quy các nhu cầu thực tế về các bài toán phát hiện giao nhau, phần bù khoảng, sắp xếp topo, lập lịch khoảng có trọng số và phân hoạch khoảng; đã lập luận tính đúng đắn và đánh giá độ phức tạp từng thuật toán.
- **Về chương trình:** có lịch tuần, danh sách, ma trận ưu tiên, thống kê, tìm giờ rảnh, sắp xếp thứ tự và gỡ trùng lịch tự động. Kiến trúc tách lõi thuật toán khỏi giao diện.
- **Về thực nghiệm:** kết quả khớp nghiệm vét cạn trên 1.500 bộ dữ liệu ngẫu nhiên, 62 ca kiểm thử tự động đều đạt, số đo hiệu năng xác nhận ảnh hưởng của mật độ lịch và của cấu trúc dữ liệu.

Qua đề tài, em rút ra ba bài học. Thứ nhất, đánh giá độ phức tạp phải gắn với dữ liệu thực tế: cùng một thuật toán có thể gần tuyến tính hoặc gần bậc hai tùy mật độ lịch. Thứ hai, chi tiết cài đặt nhỏ như chọn cấu trúc hàng đợi có thể làm thay đổi bậc độ phức tạp. Thứ ba, kiểm thử bằng cách đối chiếu với lời giải vét cạn đáng tin hơn nhiều so với vài ví dụ thủ công.

2. Hướng phát triển

- **Thêm hạn chót và tự xếp lịch:** người dùng chỉ nhập thời lượng và hạn chót, chương trình tự xếp vào lịch.
- **Việc lặp lại và nhắc việc:** lịch học hằng tuần, thông báo trước giờ bắt đầu.
- **Cho phép chỉnh hàm trọng số** và so sánh nhiều phương án dời việc, tối ưu tổng độ trễ.
- **Đồng bộ lịch:** nhập, xuất theo chuẩn iCalendar (RFC 5545) để dùng cùng các ứng dụng lịch khác.
- **Khảo sát người dùng** để đánh giá khách quan mức độ thân thiện của giao diện.

TÀI LIỆU THAM KHẢO

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein (2022), *Introduction to Algorithms*, 4th Edition, The MIT Press.

[2] J. Kleinberg, É. Tardos (2006), *Algorithm Design*, Pearson/Addison-Wesley (Chương 4: thuật toán tham lam, phân hoạch khoảng; Chương 6: lập lịch khoảng có trọng số).

[3] J. L. Bentley, T. A. Ottmann (1979), "Algorithms for Reporting and Counting Geometric Intersections", *IEEE Transactions on Computers*, C-28(9), pp. 643–647.

[4] S. R. Covey (1989), *The 7 Habits of Highly Effective People*, Free Press.

[5] Python Software Foundation, *The Python Standard Library* (`bisect`, `heapq`, `collections`, `tkinter`, `sqlite3`, `unittest`), https://docs.python.org/3/library/

[6] Python Wiki, *TimeComplexity*, https://wiki.python.org/moin/TimeComplexity

[7] A. B. Kahn (1962), "Topological Sorting of Large Networks", *Communications of the ACM*, 5(11), pp. 558–562.

[8] SQLite Documentation, *SQLite Foreign Key Support*, https://www.sqlite.org/foreignkeys.html

PHỤ LỤC: HƯỚNG DẪN CHẠY CHƯƠNG TRÌNH

Mã nguồn: https://github.com/phuhung2805/ktltnc-. Chương trình cần Python 3 có kèm Tkinter (đã kiểm thử trên Python 3.12), không cần cài thêm thư viện. Mở cửa sổ dòng lệnh tại thư mục dự án và chạy các lệnh sau:

Bảng 15. Các lệnh chạy chương trình và thực nghiệm

Lệnh	Tác dụng
`python main.py`	Mở chương trình. Lần đầu lịch trống, chọn "Nạp dữ liệu mẫu" để xem thử
`python demo_experiments.py`	Chạy các thuật toán trên dữ liệu mẫu, in kết quả ở mục 4.3
`python experiments.py`	Chạy các phép đối chiếu và so sánh ở mục 4.4, 4.5.2 đến 4.5.5
`python benchmark.py`	Đo hiệu năng theo quy mô dữ liệu ở mục 4.5.1, 4.5.5
`python -m unittest discover -s tests -t .`	Chạy toàn bộ kiểm thử tự động

Cấu trúc thư mục chính:

```
core/
  models.py            Lớp Task và các kiểu liệt kê
  scheduler_engine.py  Các thuật toán
  storage.py           Lưu trữ SQLite, nhập xuất JSON
ui/
  main_window.py       Cửa sổ chính
  calendar_view.py     Lịch tuần
  task_dialog.py       Hộp thoại thêm, sửa công việc
  smart_slot_dialog.py Tìm giờ rảnh
  eisenhower_view.py   Ma trận ưu tiên
  analytics_view.py    Thống kê
  planning_views.py    Thứ tự làm, khi trùng lịch
  widgets.py, theme.py Thành phần và kiểu dáng dùng chung
tests/                 Kiểm thử tự động
main.py, demo_experiments.py, experiments.py, benchmark.py
```